

Trabajo Práctico Integrador: Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica

Integrantes: Cortes Maria Pia, Fehler Valentina, Delahaye Augusto, Rodriguez Santos, Zerpa Santiago

Asignatura: Desarrollo de Software

Modalidad: en grupo

Criterio de Evaluación y Política de IA: Se permite el uso de herramientas de Inteligencia Artificial (LLMs). Sin embargo, el práctico está diseñado para evaluar el **razonamiento crítico, el análisis de estados de memoria y la justificación técnica**. Las respuestas genéricas generadas por IA sin prueba de trazabilidad manual o sin responder a los escenarios de borde específicos serán penalizadas.

Criterios Generales de Entrega

- Entregar un informe en formato PDF publicado en un repositorio de GitHub.
- Incluir fragmentos de código fuente en Java y los diagramas/trazas explicativas solicitadas.

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

```
import java.util.*;
class TarjetaAcceso { // clase Tarjeta de Acceso
    private String codigo; // declara un atributo llamado código tipo String
    private int nivelAcceso; // atributo tipo entero
    public TarjetaAcceso(String codigo, int nivelAcceso) { // constructor
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }
    public void setNivelAcceso(int nivelAcceso) { // método set
        this.nivelAcceso = nivelAcceso;
    }
    @Override // sobreescritura de metodos
    public boolean equals(Object o) { // sobrescribe equals
        if (this == o) return true; // si son el mismo objeto en memoria, son iguales
        if (o == null || getClass() != o.getClass()) return false; // si es null o de otra
        // clase, no son iguales
        TarjetaAcceso tarjeta = (TarjetaAcceso) o; // se castea el objeto para comparar sus
        // atributos
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
            tarjeta.codigo); // compara los atributos del objeto para ver si son
        // iguales, igualdad lógica
    }
    @Override
    public int hashCode() { // sobrescribe hascode
        return Objects.hash(codigo, nivelAcceso); // usa el método especificado en los
        // apuntes con el Objects.hash y los atributos, es para generar un hashcode o entero seguro
    }
}
import java.util.*;
public class ControlAcceso {
    public static void main(String[] args) { // define el main
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>(); // crea un HashSet
        TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2); // crea un nuevo objeto del tipo
        // TarjetaAcceso con los atributos ACC-101 y 2 como nivel de acceso
        tarjetasValidas.add(t1); // agrega el objeto al HashSet creado
        // Modificación posterior a la inserción
        t1.setNivelAcceso(5); // se genera un problema, porque se modifica el atributo
        // después de haber ingresado el objeto dentro del HashSet. Al cambiar o mutar el objeto su
        // hashcode cambia pero no cambia dentro del HashSet donde sigue ocupando el mismo lugar de
```

memoria. Ahora si se realiza alguna operación no se busca en el espacio que realmente ocupa el objeto si no en uno nuevo que corresponde al nuevo hashcode generado después de la mutación

```
System.out.println("¿Tarjeta válida en sistema?: " +
    tarjetasValidas.contains(t1)); // por la razón anterior al comprobar esto va
a devolver false ya que busca en una nueva ubicación la cual esta vacia aunque el objeto si
este almacenado en el hashcode, es como si se buscara en el lugar 5 mientras el objeto esta
almacenado en el 2, para el HashSet es como buscar en un bloque vacío
System.out.println("Cantidad de tarjetas registradas: " +
    tarjetasValidas.size()); // se puede comprobar que el HashSet no esta vacío
ya que al ejecutar este comando devuelve 1, el objeto guardado al inicio, pero ahora no se
puede acceder a este
    }
}
```

Responder y Justificar:

1. Trazo del Estado Interno: Explique paso a paso qué ocurre dentro de la estructura HashSet:

- Al ejecutar **tarjetasValidas.add(t1)**.
 - El HashSet guarda los elementos utilizando su hashCode para determinar su ubicación, pero sin mantener un orden específico. En este caso, calcula el código del objeto t1 basándose en sus atributos iniciales y lo ingresa dentro de un bucket (espacio) específico del HashSet.
- Al ejecutar **t1.setNivelAcceso(5)**.
 - Con este comando se llama al método set del objeto t1 (que es del tipo TarjetaAcceso) para cambiar el atributo denominado nivelAcceso. Si bien puede parecer que no hay problema al hacerlo, el inconveniente ocurre dentro del HashSet: el objeto se había guardado teniendo en cuenta su hashCode original. Como se sobrescribieron los métodos equals y hashCode utilizando este atributo, al mutar el nivel de acceso también se cambia el hashCode del objeto. En consecuencia, el objeto queda alojado en una ubicación (bucket) que ya no coincide con su nuevo código generado.
- Al ejecutar **tarjetasValidas.contains(t1)**.
 - Al intentar comprobar si la tarjeta existe, el HashSet vuelve a calcular el hashCode de t1. Como el nivel ahora es 5, este nuevo cálculo genera un número de ubicación totalmente distinto. El HashSet va a buscar a ese nuevo espacio, lo encuentra vacío y devuelve false. Así, el HashSet no está vacío, ya que el objeto t1 sigue ocupando su bucket original, pero resulta imposible acceder a él, generando un bloque ocupado que quedó "perdido" en la memoria.

2. Explicación del Cajón (Bucket): ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer contains()?

El objeto quedó guardado en la ubicación o bucket correspondiente al cálculo del hashCode realizado con los atributos originales del objeto, siendo su código "ACC-101" y su nivel de acceso 2. Pero Java no lo busca en este bucket al hacer **contains()**, ya que al mutar el atributo de nivel de acceso (cambiando el 2 por un 5), el hashCode se recalcula y da un resultado diferente a la ubicación en la que realmente quedó guardado el objeto.

Para comprobarlo, agregamos comandos para imprimir en consola el valor de **t1.hashCode()** antes y después de la modificación. La salida del programa demuestra claramente cómo el código hash cambia, provocando que la búsqueda devuelva false:

```
HashCode original: -1387020803
¿Tarjeta válida en sistema?: false
Cantidad de tarjetas registradas: 1
HashCode mutado: -1387020800
```

3. Fuga de Memoria (Memory Leak): ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

Se genera una fuga de memoria por dos razones principales: así como el método **.contains(t1)** regresa **false** al buscar el objeto en una ubicación o bucket equivocado (debido a lo explicado en el punto anterior), es imposible eliminar el objeto utilizando un **.remove(t1)**, ya que este intentaría remover el objeto en una ubicación que dentro del HashSet se encuentra vacía. Además, el **Garbage Collector** no elimina el objeto porque, aunque sea inaccesible para nosotros, el HashSet sigue manteniendo una referencia interna hacia él en su bucket original; por lo tanto, el recolector asume que el objeto sigue en uso y no lo libera de la memoria.

Al tratarse de un servidor que opera 24/7 y que se ejecuta repetidamente, el HashSet comenzará a llenarse de estas referencias retenidas, acumulando objetos que podríamos llamar "fantasmas". Si bien al usar un **.size()** se podrá observar que el HashSet sigue creciendo, no será posible acceder a ninguno de estos objetos. De esta manera, se consumirá progresivamente toda la memoria RAM asignada, lo cual terminará llevando a la fuga de memoria y a problemas graves como el colapso del servidor.

4. Refactorización Obligatoria: Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

Primera Solución

```
import java.util.*;
class TarjetaAcceso {
    private final String codigo;
    private final int nivelAcceso;
    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    public String getCodigo() {
        return codigo;
    }
    public int getNivelAcceso() {
        return nivelAcceso;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
            tarjeta.codigo);
    }
    @Override
    public int hashCode() {
        return Objects.hash(codigo, nivelAcceso);
    }
}

import java.util.*;
public class ControlAcceso {
    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();
        TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);
        tarjetasValidas.add(t1);
        // Modificación posterior a la inserción
        t1.setNivelAcceso(5); // Hay un error en esta línea ya que se han eliminado los
        metodos set
        System.out.println("¿Tarjeta válida en sistema?: " +
            tarjetasValidas.contains(t1));
        System.out.println("Cantidad de tarjetas registradas: " +
            tarjetasValidas.size());
    }
}
```

Esta solución prohíbe que el objeto cambie de estado una vez que fue creado. Como el objeto no puede cambiar, el hashCode tampoco va a cambiar y de esta forma no se perderá dentro del HashSet.

Se eliminaron todos los métodos set, agregamos unos métodos get aunque realmente no se utilicen y los atributos son marcados como final para evitar que sean mutados. Si se desea cambiar una tarjeta se tendría que eliminar la actual y crear una nueva.

Segunda Solución

```
import java.util.*;
class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;
    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }
    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return Objects.equals(codigo, tarjeta.codigo); // Solo se compara el código
    }
    @Override
```

```

    public int hashCode() {
        return Objects.hash(codigo);
    } // El cálculo del bucket solo depende del código. Como el código no cambia,
    el hash será siempre el mismo aunque mute el nivel.
}
import java.util.*;
public class ControlAcceso {
    public static void main(String[] args) {
        Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();
        TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);
        tarjetasValidas.add(t1);
// Modificación posterior a la inserción
        t1.setNivelAcceso(5);
        System.out.println("¿Tarjeta válida en sistema?: " +
            tarjetasValidas.contains(t1));
        System.out.println("Cantidad de tarjetas registradas: " +
            tarjetasValidas.size());
    }
}

```

Esta solución aborda el problema desde el diseño lógico del negocio. Consideramos que la tarjeta de acceso está dada unívocamente por su código (ej. "ACC-101"), mientras que el "nivel de acceso" es solo una característica temporal que puede variar sin que la tarjeta deje de ser físicamente la misma. Para implementarlo, se mantiene la mutabilidad (se deja el *setter*), pero se modifican los atributos afectados por *equals()* y *hashCode()* para que exclusivamente evalúen el atributo código, ignorando cualquier cambio en el nivel.

Con las correcciones se obtiene la siguiente salida:

¿Tarjeta válida en sistema?: true

Cantidad de tarjetas registradas: 1

De esta manera podemos comprobar que las soluciones funcionaron, ya que el *.contains(t1)* devuelve *true*, lo que indica que es posible localizar la ubicación del objeto dentro del *HashSet*.

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Diseñe el backend de un módulo para una plataforma de *E-Commerce*. Debe elegir la interfaz y la implementación concreta del Java Collections Framework (*java.util*) más eficiente para cada una de las siguientes tres funcionalidades:

Requerimiento 1: Sistema de Auditoría de Transacciones

Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.

Requerimiento 2: Catálogo de Ofertas Relámpago

Mantener los productos ordenados automáticamente por precio de menor a mayor. Debe permitir consultar eficientemente el producto más barato y el más caro.

Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos

Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.

Responder y Justificar:

Completar la siguiente tabla comparativa y justificar la decisión con la complejidad algorítmica Big O:

Requerimiento	Interfaz Seleccionada (List , Set , Map , Queue) PDF	Implementación Concreta (ArrayList , TreeSet , HashMap , etc.) PDF	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
1. Auditoría					
2. Ofertas					
3. Sesiones					

Requerimiento	Interfaz Seleccionada	Implementación Concreta	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de la Elección
Auditoría	List	ArrayList	Si se accede a un elemento de una posición concreta $O(1)$ $O(1)$ por índice $O(n)$ secuencial	Insertar o eliminar un elemento en el centro es $O(n)$, porque requiere desplazar todos los elementos posteriores. Si se quiere insertar o eliminar al final, sería $O(1)$, ya que no debe desplazar elementos. $O(1)$ amortizado al final de la lista $O(n)$ si fuera en medio	Se seleccionó List porque necesitamos una estructura que conserve el orden exacto de inserción. Para la implementación concreta, elegimos ArrayList en lugar de LinkedList porque almacena los elementos en posiciones continuas de memoria. Con ArrayList es posible insertar miles de registros al final de la colección, en tiempo constante $O(1)$, ya que no necesita desplazar elementos existentes. Descartamos LinkedList no porque su inserción al final sea más lenta (también es $O(1)$, ya que mantiene una referencia directa al último nodo), sino por dos razones: primero, cada elemento de LinkedList requiere crear un objeto Node adicional con referencias al nodo anterior y al siguiente, lo cual consume más memoria que ArrayList. Segundo, al recorrer la colección secuencialmente para las auditorías, ArrayList aprovecha la localidad de memoria (los datos están uno al lado del otro, y puede traerlos en bloque a caché), lo que lo hace más rápido que ir entre nodos dispersos en memoria como hace LinkedList (aunque ambas tengan la misma complejidad $O(n)$ para el recorrido completo).
Ofertas	Set (SortedSet y NavigableSet)	TreeSet	$O(\log n)$	$O(\log n)$	Este requerimiento exige mantener los productos ordenados y poder consultar de forma eficiente cuál es el más barato y cuál es el más caro. Elegimos TreeSet porque implementa SortedSet, lo que garantiza que los elementos se mantengan ordenados de forma ascendente, ya sea por su orden natural o mediante un Comparator. Además, también funciona como NavigableSet, incorporando métodos de búsqueda por proximidad, como lower() para obtener un elemento menor y higher() para obtener uno mayor. También permite obtener directamente el producto más barato con first() y el más caro con last(),

					siempre que el orden esté basado en el precio. Así, esta estructura mantiene los elementos siempre ordenados y permite navegar entre ellos de manera eficiente, organizándolos internamente mediante un árbol binario auto-balanceado, conocido como árbol Rojo-Negro.
Sesiones	Map	HashMap	O(1) promedio O(n) peor caso	O(1) promedio O(n) peor caso	Se seleccionó Map para guardar las sesiones como pares clave-valor, donde cada sesión puede identificarse mediante una clave única, en este caso con su token. Para la implementación concreta, elegimos HashMap porque utiliza una tabla hash para almacenar las entradas y permite realizar operaciones rápidas como put, get y remove en O(1) promedio. Además, no permite claves duplicadas, lo cual es importante para que una misma sesión no quede registrada más de una vez con la misma clave. No garantiza un orden, pero en este caso no es un problema, ya que lo principal es poder acceder rápidamente a una sesión específica. En el peor caso, si se producen muchas colisiones (varias claves caen en el mismo bucket, entonces se debe revisar dentro para encontrar la correcta con equals()), la búsqueda o inserción podría llegar a O(n) debido a la comparación contra varios elementos.

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

```
public static Character primerCaracter noRepetido(String texto) {
    // Algoritmo ineficiente O(n^2)
    for (int i = 0; i < texto.length(); i++) { char c =
        texto.charAt(i);
        boolean duplicado = false;
        for (int j = 0; j < texto.length(); j++) { if (i != j && c
            == texto.charAt(j)) {
                duplicado = true; break;
            }
        }
        if (!duplicado) return c;
    }
    return null;
}
```


Responder y Justificar:

1. **Refactorización a $O(n)$:** Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).

```
import java.util.HashMap;
import java.util.Map;
public class ej3 {
    public static Character primerCaracterNoRepetido (String texto){
        // Creamos un HashMap, que funciona almacenando los datos en pares de clave-valor.
        // En este caso, la clave será el Character (cada letra de la palabra),
        // y el valor asociado a esa clave será el Integer (la cantidad de veces que
        aparece).
        Map<Character, Integer> frecuencia = new HashMap<Character, Integer>();

        // Primer paso: se usa un for para recorrer toda la palabra,
        // guardando cada carácter junto con la cantidad de veces que aparece en la misma
        for(int i=0; i<texto.length(); i++){
            //Se le asigna a la variable "c" el carácter que se encuentra en la posición i
            char c = texto.charAt(i);
            // Se utiliza una variable para consultar y guardar la cantidad de veces que ya
            apareció este carácter (su "valor" actual)
            Integer cantidadActual = frecuencia.get(c);

            // Si es la primera vez que aparece el carácter, se registra en el HashMap con
            una cantidad = a 1
            if(cantidadActual == null){
                frecuencia.put(c,1);
                // Si ya existía la clave (letra), se actualiza su valor sumándole 1 a su
                cantidad anterior
            }else{
                frecuencia.put(c, cantidadActual + 1);
            }

        }

        // Segundo paso: for que no está anidado para evitar una complejidad de Orden 2,
        logrando dos recorridos lineales independientes
        for(int i=0; i<texto.length(); i++){
            // Nuevamente, se asigna a la variable "c" el carácter que está en la posición
            i
            char c = texto.charAt(i);
            // Verifica si la frecuencia (valor) de este carácter (clave) es exactamente
            igual a 1
            if(frecuencia.get(c)==1) {
                return c;
            }
        }
        return null;
    }
}
// Se utiliza HashMap y no otra colección que mantenga el orden (como LinkedHashMap),
// ya que no recorreremos la colección para buscar el resultado.
// En su lugar, volvemos a evaluar la palabra original de izquierda a derecha.
```

2. **Análisis de Colisiones:** Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Internamente, un **HashMap** funciona como un gran arreglo que organiza la información en posiciones llamadas buckets. Al ingresar una clave, Java utiliza el método **hashCode()** para generar un número entero que actúa como índice y determina en qué bucket se debe guardar el par clave-valor.

Según las reglas del lenguaje, si dos objetos son iguales según el método **equals()**, obligatoriamente

comparten el mismo **hashCode()**; sin embargo, puede ocurrir lo contrario: dos objetos distintos pueden generar el mismo valor hash y caer en el mismo índice (que dos objetos tengan el mismo hash no indica que son iguales), provocando lo que se conoce como una [colisión de hash](#).

Para evitar sobrescribir el dato anterior y perder información, la estructura no guarda un elemento por posición o bucket, sino el nodo inicial de una lista enlazada. Se ejemplifica cómo el **HashMap** resuelve esto en la línea **frecuencia.put(c, cantidadActual + 1)** del código anterior. Cuando llega una letra, como por ejemplo la 'a', el sistema calcula su **hashCode()** y va al bucket correspondiente en el arreglo. Si ya hay elementos guardados en esa posición, no los pisa; en su lugar, comienza a recorrer la lista enlazada comparando la clave nueva con las ya existentes utilizando el método **.equals()** (evaluando, por ejemplo, si **'a'.equals('x')**). Si este método devuelve true, significa que es la misma letra, por lo que el **HashMap** simplemente sobrescribe el valor, permitiendo que se actualice la frecuencia sumándole 1 sin duplicar el carácter. Si devuelve false, entiende que son caracteres distintos que casualmente colisionaron (con el mismo hashCode), y simplemente enlaza la nueva letra al final de la lista para no perderla.

El proceso de búsqueda, cuando se usa la instrucción **frecuencia.get(c)** en el segundo bucle, usa la misma lógica. Primero, calcula el **hashCode()** de la letra que se busca y va directo a su bucket. Si en ese bucket hay varios elementos encadenados debido a colisiones previas, los recorre uno por uno aplicando **.equals()** a cada elemento contra la letra que se busca. En el momento exacto en que **.equals()** devuelve true, el sistema sabe que encontró el carácter que se buscaba y devuelve, en este caso, su frecuencia correspondiente (el valor Integer), ignorando por completo al resto de los elementos guardados allí. En otras palabras, busca con la clave y devuelve el valor.

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que **s1.hashCode() == s2.hashCode()** devuelve true (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un `HashSet<String>`, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos `hashCode()` y `equals()` internamente para determinar si `s2` debe ingresarse o desecharse.

Si se insertan `s1` y `s2` en un `HashSet<String>`, la colección resultante contendrá dos elementos, esto se debe a que, si bien ambos objetos poseen el mismo `hashCode()` y por lo tanto son dirigidos al mismo bucket de la tabla interna, Java no se basa únicamente en el hash para determinar si un elemento ya existe. Al coincidir los `hashCode()`, se invoca adicionalmente el método `equals()` para confirmar si se trata realmente del mismo objeto lógico. Como `s1.equals(s2)` retorna false (las cadenas "FB" y "Ea" son distintas), Java concluye que `s2` no es un duplicado y lo inserta como un nuevo elemento dentro del mismo bucket, enlazado a `s1`. En definitiva, el `hashCode()` igual solo indica que dos objetos podrían ser iguales, mientras que `equals()` es el que determina si efectivamente lo son.

El método `hashCode()` cumple una función de localización (determina en qué bucket debe buscarse o insertarse el elemento), mientras que el método `equals()` cumple la función de decisión (determina si dos objetos son lógicamente equivalentes). Por esta razón, el `HashSet` finaliza con dos elementos: "FB" y "Ea" comparten el mismo bucket, pero son reconocidos como entidades distintas.

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un `HashMap` en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?

La acumulación de colisiones de hash puede afectar el rendimiento de un `HashMap`. En el caso ideal, las claves se distribuyen en distintos buckets, permitiendo que las operaciones de búsqueda, inserción y eliminación tengan una complejidad promedio de $O(1)$.

Si muchas claves diferentes generan el mismo `hashCode()` y terminan en el mismo bucket, Java debe comparar los elementos que se encuentran allí utilizando `equals()`. Si se acumulan muchos elementos en ese mismo bucket, en el peor caso la búsqueda puede llegar a ser $O(n)$, ya que debe revisar los elementos uno por uno.

Desde Java 8, cuando un bucket acumula una cantidad importante de elementos, Java puede convertir su estructura en un árbol rojo-negro, haciendo que la búsqueda dentro de ese bucket pueda realizarse en $O(\log n)$.

Por lo tanto, mientras que un `HashMap` tiene un rendimiento promedio de $O(1)$, una gran cantidad de colisiones puede hacer que pase a $O(n)$ en el peor caso, o $O(\log n)$ cuando se utiliza el árbol rojo-negro.